Traffic Flow Prediction System

Hoàng Minh Đức

Nguyễn Hoàng Tuấn Kiệt

Why Traffic Flow?

- Traffic flow Prediction is crucial for:
  - Traffic planning in navigation apps such as Google, which helps urban planning
  - Route planning, for navigation
  - Public transport planning
  - Logistics and Delivery

Data and methodology

- In this process, we are using the Boroondara 2006 data to train a model that will predict the traffic flow in various sites.
- To achieve this, first we process the data to result in a matrix. This matrix contains the Flow Information: every row is a location, every column is a timestamp.

(More detailed data processing in code…)

Data

The data in this process include:

- Sensor SCATS region number, Road names, Latitude and Longitude
- Actual flow data in 15-minute intervals from Oct 1 2006 to Oct 31 2006
- Some various categorical data.

Data processing and Model approach

- To process the data, first We seperated all of the sensors using an Identifier; a combination of the HF Vicroads Internal value and the SCATS region value.
- We transform the dataset to include other data via-identifier (lat, long, flow, etc)
- We flatten the flow so it becomes a single row with 15-minute sequential data.
- In order to help create windows for later, we transpose the dataset.

We will use a time-series LSTM Model approach to solve this problem.

Data after processing

	0970 - 10503	0970 - 16116	0970 - 249	0970 - 5887	2000 - 10505	2000 - 12521	2000 - 2276	2000 - 6246	2200 - 10074	2200 - 12428	...	4324 - 14646	4324 - 16859	4324 - 6621	4812 - 16535	4812 - 4250	4812 - 6302	4821 - -1	4821 - 14527	4821 - 16911	4821 - 6673
0	92	37	86	47	110	94	115	46	25	38	...	32	43	28	58	64	49	0	25	44	86
1	93	30	83	42	93	82	99	41	17	49	...	29	34	25	53	63	54	0	28	50	86
2	90	26	52	31	83	49	94	42	16	32	...	32	35	43	57	83	48	0	21	49	89
3	55	29	58	34	53	64	86	36	21	24	...	15	24	30	43	68	50	0	15	42	74
4	64	14	59	40	58	48	75	26	18	20	...	25	19	19	39	64	31	0	16	31	73
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2971	98	38	79	0	0	0	0	0	17	31	...	39	52	46	0	0	0	0	35	68	87
2972	88	30	71	0	0	0	0	0	24	28	...	23	24	36	0	0	0	0	29	44	75
2973	54	16	55	0	0	0	0	0	17	19	...	28	30	28	0	0	0	0	24	35	63
2974	71	17	45	0	0	0	0	0	14	14	...	22	19	27	0	0	0	0	25	30	51
2975	66	14	33	0	0	0	0	0	8	18	...	13	23	18	0	0	0	0	14	28	54

Windows

- After having the data, we create time-series windows. Each window is 1 week long (672 15-minute intervals) of data.
- We set the horizon=24 to predict 24 15-minute intervals (5 hours) ahead.
- We create two window arrays: X for all values but last HORIZON steps, y for all next HORIZON steps. X is split into X_train, X_val, X_test respectively, as well as y_train, y_val, y_test for y

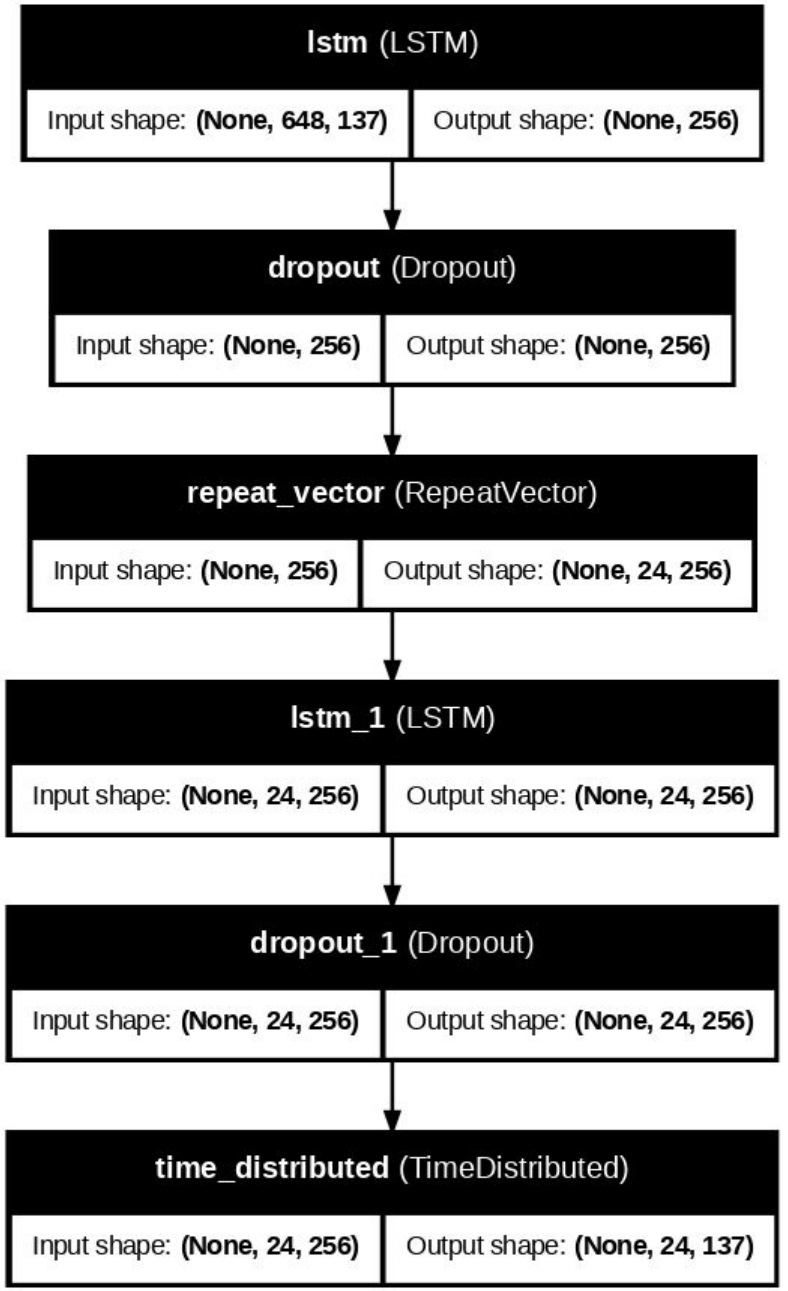
Model approaches

- We train multiple-location LSTM and GRU models to predict the data over all of the locations.
- This approach allows locations that are close geographically to be able to learn pattern from others.

Model Architecture - LSTM

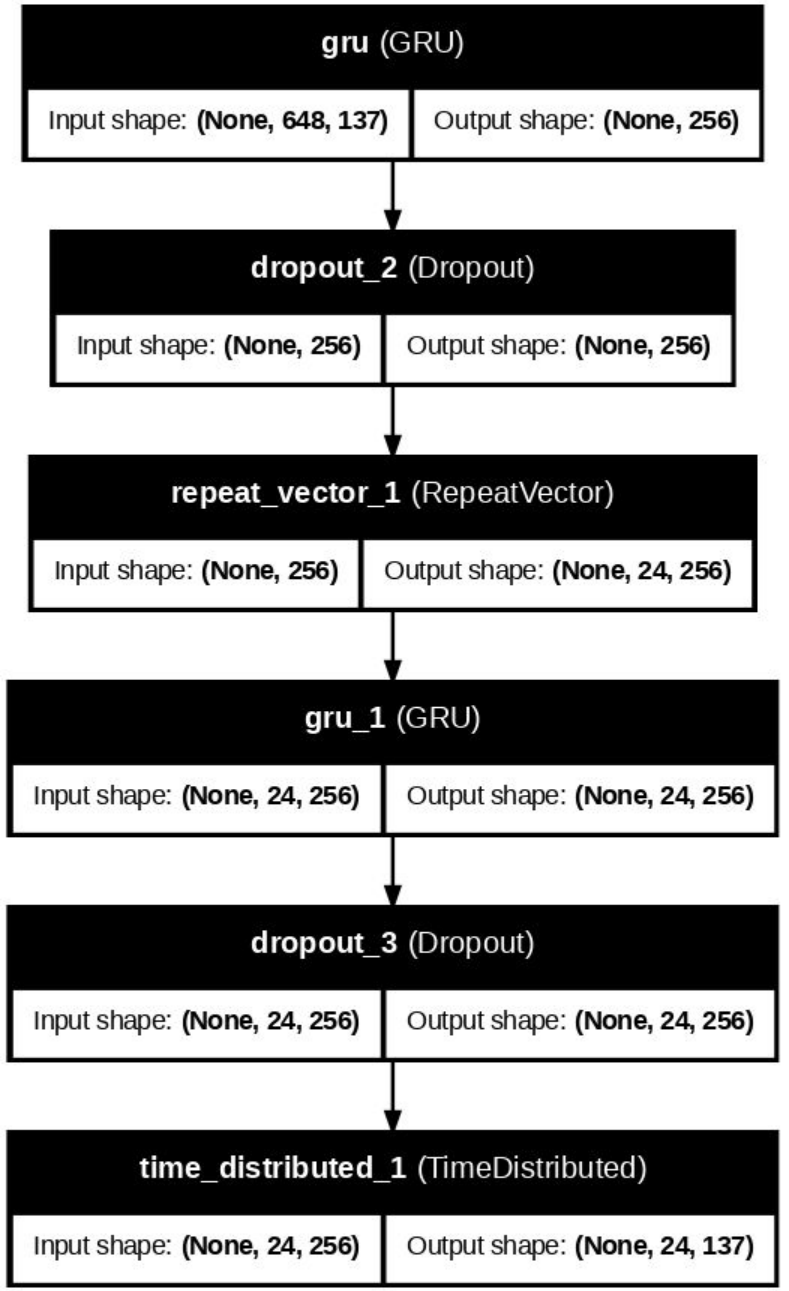
The LSTM model has the following layers that follow an encoder-decoder architecture:

- Input: receive input (not shown in plot)
- First LSTM layer: Process the sequence and compresses it to a 256-dimensional context vector
- First Dropout: Regularization, zeros out 20% of input to not overfit
- RepeatVector: Repeat the context vector 24 times for the decoder to use as initial input
- Second LSTM: Takes the repeated context vector and processes the information required to predict the time step
- Second dropout: similar regularization
- TimeDistributed: Convert hidden states to final predictions



Model Architecture - GRU

The GRU model has a similar architecture to the LSTM model, with the LSTM layers replaced by the GRU layers due to their similar functionality.



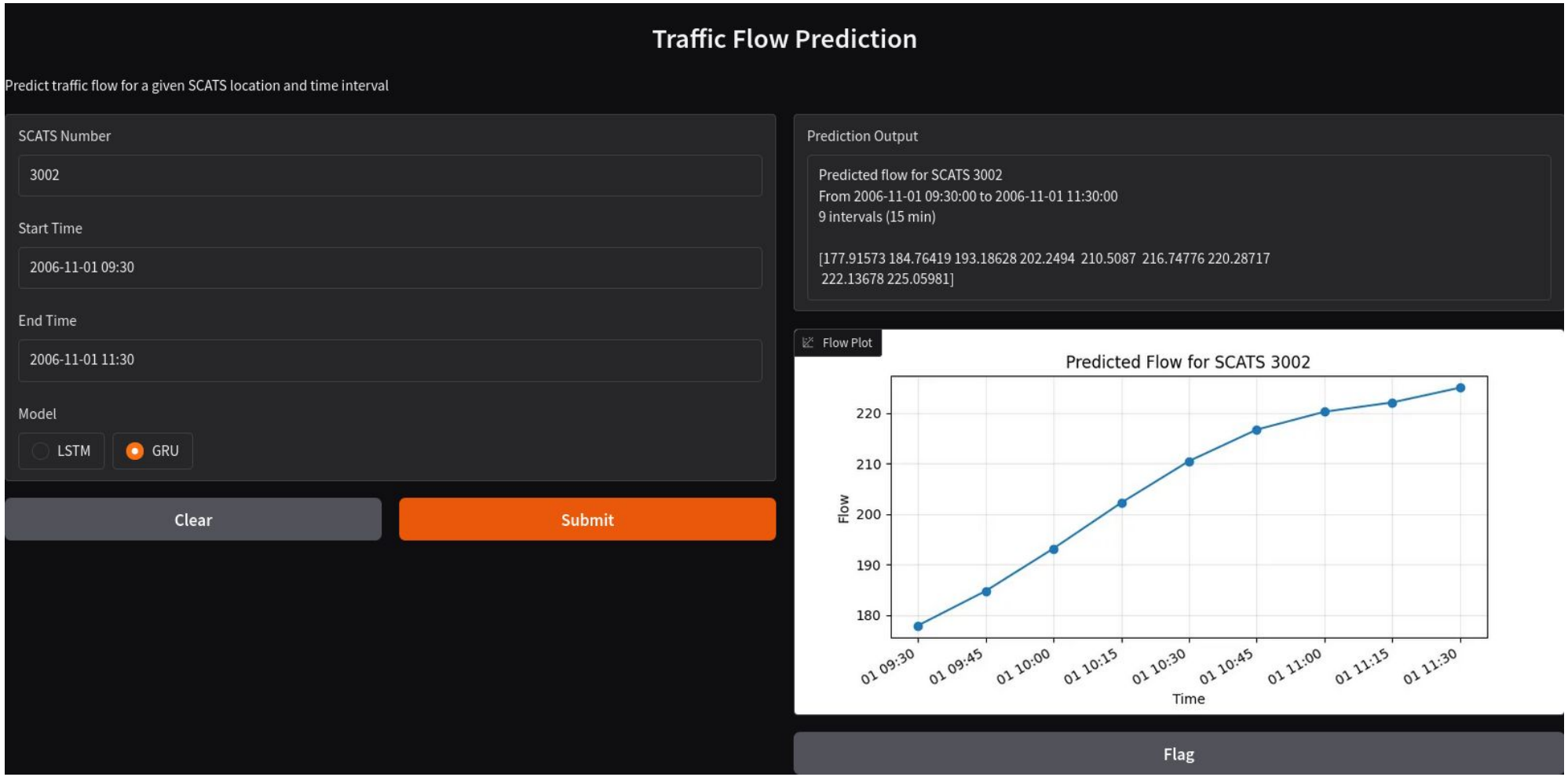
Training

- We split the data into groups of train/val/test with ratios 70-15-15
- Before training, we normalize the data for the neural network using a Standard Scaler. (zero mean, unit variance).
- Both models use AdamW as their optimizer, as well as having EarlyStopping (stop training when loss doesn’t change much) and ReduceLROnPlateau (reduce learning rate to keep training) callbacks.
- Models are trained on normalized data. Therefore, predictions are normalized, and a denormalization is required to obtain real values.
-

Prediction results

Metric	Normalized		Denormalized	
	LSTM	**GRU**	**LSTM**	**GRU**
MAE	0.394569	0.356201	34.609779	31.244328
MSE	0.504611	0.406004	3882.469204	3123.785721
RMSE	0.710360	0.637184	62.309463	55.890838
R2	0.448813	0.556522	0.448813	0.556522

UI


Traffic Flow Prediction
Predict traffic flow for a given SCATS location and time interval
SCATS Number
3002
Start Time
2006-11-01 09:30
End Time
2006-11-01 11:30
Model
LSTM
GRU
Clear
Submit
Prediction Output
Predicted flow for SCATS 3002
From 2006-11-01 09:30:00 to 2006-11-01 11:30:00
9 intervals (15 min)
[177.91573 184.76419 193.18628 202.2494 210.5087 216.74776 220.28717
222.13678 225.05981]
Flow Plot
Predicted Flow for SCATS 3002
Flow
220
210
200
190
180
01 09:30
01 09:45
01 10:00
01 10:15
01 10:30
01 10:45
01 11:00
01 11:15
01 11:30
Time
Flag

Limitations and Future work

- Low accuracy. The MAE and RMSE is currently high, which we will mitigate using hyperparameter tuning, as well as discover other models.
- Needed routing future for GUI
- Adapt to different datasets.

Thanks for Listening!